

# Project 1 Pitch: Crew Job Tracker

Jose Rayos · Flatiron School Software Engineering Capstone (Course 2531) · Project 1: Productivity App  
Pitch and MVP discussion due Oct 10, 2026 · Final app and showcase due Oct 17, 2026

## 1. Problem, Users, and Value

**Business problem.** For three years I ran Crystal Clear Windows and Janitorial, a cleaning business with 10 employees. Scheduling and payment tracking lived on paper and in text messages. It was hard to see the week's jobs in one place, and easy to lose track of which completed jobs had not been paid. Crew Job Tracker is the simple tool I wish I had then.

**Target users.** Owners of small cleaning or service businesses (window cleaning, janitorial, lawn care, handyman) who manage their own schedule and billing and don't need a large field-service platform.

**Value.** One place to keep client details, schedule jobs, mark them completed or cancelled, and see at a glance what is coming up this week and what is still unpaid.

### User stories

1. As an owner, I can register, log in, and log out so my business data stays private.
2. As an owner, I can add, view, edit, and delete clients (name, address, phone, notes).
3. As an owner, I can schedule a job for a client with a date, time, service type, price, and notes.
4. As an owner, I can update a job's status (scheduled, completed, cancelled) and mark it paid or unpaid.
5. As an owner, I can open my dashboard and see this week's schedule in date and time order.
6. As an owner, I can see a list of unpaid jobs so I know who to follow up with.
7. As an owner, I can open a client and see their full job history.
8. As an owner, I can only see and change my own clients and jobs, never another user's.

**Optional API/AI (stretch, after MVP).** An "Open in Google Maps" link built from the client's address (a plain URL, no API key), and possibly an AI summary of a client's job notes.

## 2. Development Plan

### Process (5 steps)

1. **Plan and model:** finalize user stories, models, and routes; set up the repo, Flask app, and Vite app.
2. **Back end:** SQLAlchemy models, Flask-Migrate migrations, seed data, marshmallow schemas, and RESTful CRUD routes for clients and jobs; test with Postman.
3. **Auth and ownership:** register/login/logout/check-session with Flask-Bcrypt and session auth; every client and job query is filtered by the logged-in user, and requests for another user's records return 404.
4. **Front end:** React Router pages, auth context, forms, and the dashboard, using fetch with loading and error states; style with Tailwind.
5. **Polish and deliver:** validation, edge cases, README, screenshots, video walkthrough, and reflection; stretch goals only if the MVP is done.

### Data models and relationships

| Model  | Key fields                                                                                                                        | Relationships                                    |
|--------|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| User   | id, username, email, password_hash                                                                                                | has many Clients; has many Jobs                  |
| Client | id, name, address, phone, notes, user_id (FK)                                                                                     | belongs to User; has many Jobs (delete cascades) |
| Job    | id, date, time, service_type, status (scheduled / completed / cancelled), price, paid (bool), notes, client_id (FK), user_id (FK) | belongs to Client; belongs to User               |

**API routes (planned):** POST /signup, POST /login, DELETE /logout, GET /check\_session; GET/POST /clients, GET/PATCH/DELETE /clients/<id>; GET/POST /jobs, GET/PATCH/DELETE /jobs/<id>; GET /dashboard (this week's jobs and unpaid jobs).

### React component plan

- App (routes) with AuthContext for the current user; NavBar; ProtectedRoute for logged-in pages

- LoginPage, SignupPage
- Dashboard → WeekSchedule, UnpaidJobsList
- ClientsPage → ClientList, ClientForm; ClientDetail → client info and its JobList
- JobsPage → JobList (filter by status/paid), JobForm
- Shared: Loading, ErrorMessage. Hooks: useState, useEffect, useContext, and useParams/useNavigate from React Router.

**Tools:** Flask, Flask-SQLAlchemy, Flask-Migrate, Flask-Bcrypt, Flask-Login (or Flask sessions), marshmallow, SQLite in development and PostgreSQL for deployment, React (Vite), React Router, fetch, Tailwind CSS, Git/GitHub, Postman. Stretch: Render deployment.

#### Rubric alignment

| Rubric area                 | How this project addresses it                                                                                                                |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Alignment with brief        | React + Flask + SQLAlchemy, three related tables, full CRUD over REST, async fetch with loading/error states, auth and ownership             |
| Functionality / auth        | Bcrypt-hashed passwords, session login, protected routes on both client and server, ownership checks on every record                         |
| Code quality / structure    | Separate <code>server/</code> and <code>client/</code> folders; models, schemas, and routes organized by resource; small reusable components |
| Documentation / Git hygiene | Public repo, feature branches, small descriptive commits; README with install, usage, routes, and screenshots                                |
| Professionalism             | Pitch and MVP on time, clear video under 10 minutes, written reflection, responsive and consistent UI                                        |

### 3. Timeline, Scope, and Risks

| Dates                   | Work                                                                                                    |
|-------------------------|---------------------------------------------------------------------------------------------------------|
| Sat Oct 3 – Sun Oct 4   | Write pitch; repo setup; Flask and Vite skeletons; models, migrations, seed data                        |
| Mon Oct 5 – Wed Oct 7   | Auth routes; client and job CRUD with ownership checks; marshmallow schemas; Postman testing            |
| Thu Oct 8 – Fri Oct 9   | React Router, auth pages, AuthContext, clients pages                                                    |
| <b>Sat Oct 10</b>       | <b>Submit pitch and post MVP discussion</b> (working auth, back-end CRUD, clients UI)                   |
| Sun Oct 11 – Tue Oct 13 | Jobs pages, status/paid updates, dashboard, loading/error states, Tailwind styling                      |
| Wed Oct 14 – Thu Oct 15 | Apply peer feedback, validation and bug fixes, README and screenshots; stretch goals if MVP is complete |
| Fri Oct 16              | Record video walkthrough (10 minutes max); write reflection                                             |
| <b>Sat Oct 17</b>       | <b>Final testing; submit app and showcase</b> (by 11:59 PM ET)                                          |

**MVP scope:** auth, client CRUD, job CRUD with status and paid tracking, dashboard with this week's schedule and unpaid jobs, ownership checks. **Out of scope for MVP:** employee accounts and crew assignment, invoicing or online payments, recurring jobs, AI and maps features.

#### Risks and mitigation

- *Session cookies between Vite and Flask:* use the Vite dev proxy and `credentials: "include"`; test auth early (step 3, first week).
- *Date and time logic for "this week":* store date and time as separate fields, define the week as Monday through Sunday, and compute it on the server.
- *Scope creep:* stretch features only start after the full MVP works end to end.
- *SQLite to PostgreSQL differences on deploy:* keep migrations clean and test against PostgreSQL before deploying.

**Research:** Flask-Login vs. plain Flask sessions, nested marshmallow schemas (client with jobs), Vite proxy setup, Render deployment with PostgreSQL, and the Google Maps search URL format.